

Módulo: Programación Multimedia y dispositivos móviles.
Unidad de Trabajo N°1: Introducción al desarrollo de aplicaciones móviles.
Actividad de Clase 9: Menu y Persistencia de datos (SharedPreferences y ficheros).

1. Objetivo general.

Conocer, manejar y generar aplicaciones que trabaje con el almacenamiento de datos.

2. Metodología.

Cada alumno realizará la app.

3. Descripción.

Introducción

Vamos a realizar una pequeña App que trabaje con la persistencia de datos. Tanto con **SharedPreferences** como con **Ficheros**. Y también vamos a hacerlos con un **Menu**. Para ello vamos a realizar una App que tiene tres Activities. Una principal con el menu y según seleccione SharedPreferences o Ficheros abre la Activity correspondiente.

Menu

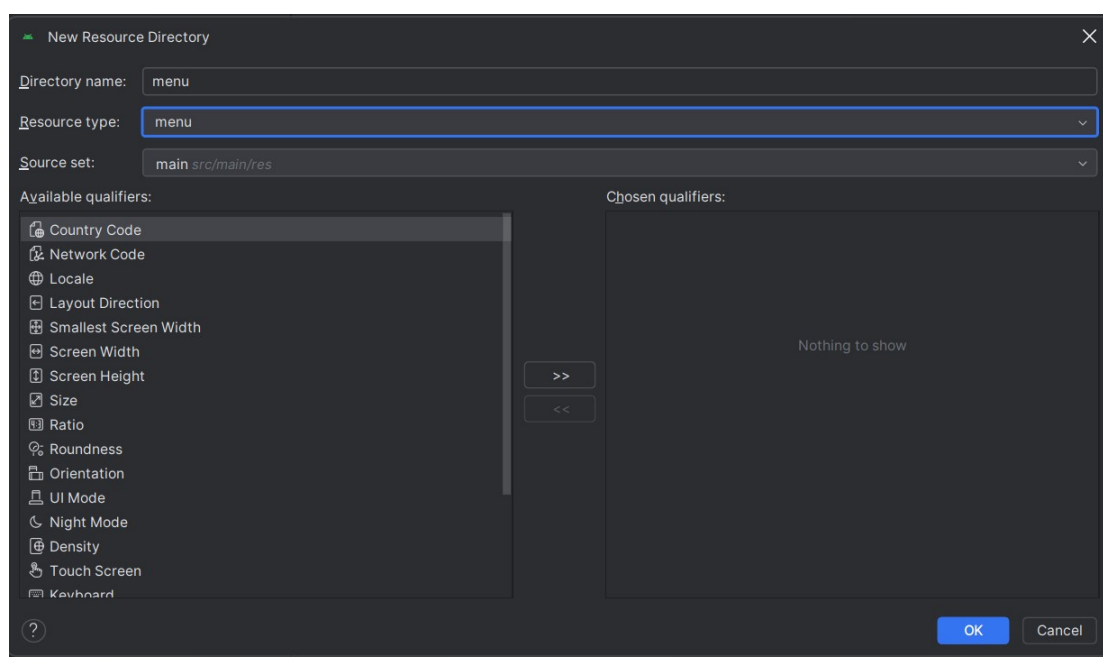
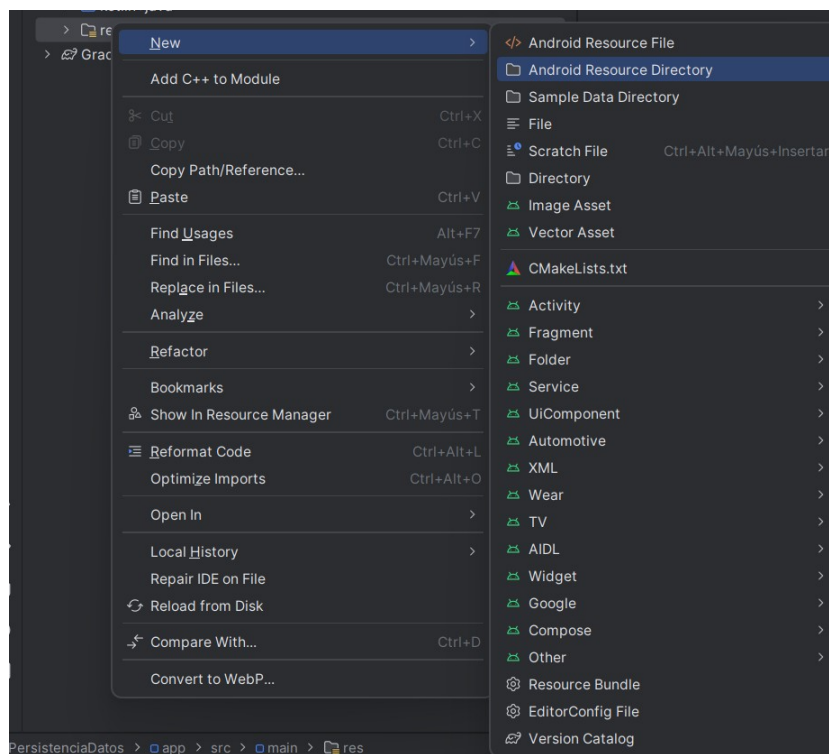
En las últimas versiones no tenemos activada la **Toolbar**, vamos a añadirla. En el fichero `activity_main.xml` añadiremos la **Toolbar**. Para ello añade lo siguiente al fichero xml en cuestión.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

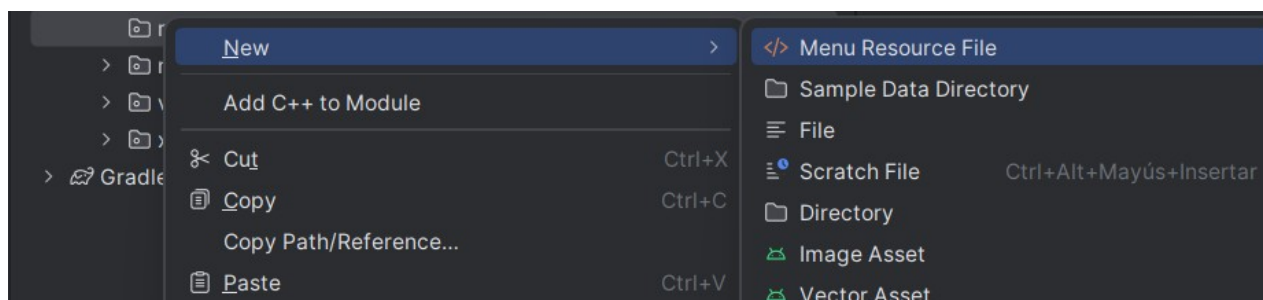
    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolb"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="?attr/colorPrimary"
        android:minHeight="?attr/actionBarSize"
        android:theme="?attr/actionBarTheme"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">
    </androidx.appcompat.widget.Toolbar>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Ahora vamos a crear un menú, en la carpeta **res** como un nuevo recurso, de tipo menú. Para ello con el botón secundario (derecho generalmente) sobre la carpeta **res** y pulsamos sobre **New** → **Android Resource Directory**



Luego creamos el XML, para ello dentro del directorio menu, creamos un Menu Resource File



Una vez creado podemos modificarlo de forma manual o usar el diseñador como hacíamos con las layouts. Añade dos items al menú uno para Preferencias y otro para ficheros. Coloca el id y el título (lo que aparecerá en el menú) que quieras. A continuación puedes ver una imagen del fichero XML con las opciones del menú con sus id y su título correspondiente.

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/preferencias"
        android:title="Preferencias" />
    <item
        android:id="@+id/ficheros"
        android:title="Ficheros" />

</menu>
```

A continuación tenemos que vincular el menu y manejar las diferentes opciones. Para ello debemos escribir el código en Kotlin que debe ser similar al siguiente:

```
override fun onCreateOptionsMenu(menu: Menu?): Boolean {  
    super.onCreateOptionsMenu(menu)  
    menuInflater.inflate(R.menu.menu, menu)  
    return true  
}  
  
val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())  
v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)  
insets  
}  
  
// Accedemos a la Toolbar  
val tool = findViewById<Toolbar>(R.id.toolbar)  
// Activamos la Toolbar  
setSupportActionBar(tool)  
}
```

Vamos con la implementación del menú. En primer lugar creamos las opciones para ello dentro de la clase MainActivity sobrescribe (override) el método `onOptionsItemSelected` donde mostramos el menú (método `inflate`)

El código kotlin para la selección del ítem puede ser similar al que aparece a continuación. Dicho código también estará dentro de nuestra clase MainActivity.

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {  
    when(item.itemId) {  
        R.id.persistencia -> {  
            preferencias()  
            return true  
        }  
  
        R.id.ficheros -> {  
            fichero()  
            return true  
        }  
    }  
    return super.onOptionsItemSelected(item)  
}
```

En dicho código hemos puesto las funciones `preferencias()` y `fichero()` que harán que abran la Activity correspondiente. A continuación puedes ver un video del funcionamiento de la App con el menú

https://drive.google.com/file/d/1nDjed1rOfz6jQ0aMcYqY5N9Z_YrL_rZM/view?usp=sharing

Ahora veremos como realizar cada una de las Activities.

Introducción a SharedPreferences.

Las preferencias no son más que datos que una aplicación debe guardar para personalizar la experiencia del usuario, por ejemplo información personal, opciones de presentación, etc. Están destinadas a almacenar pequeñas cantidades de información a través de clave-valor. Debe ser información no comprometida puesto que Android generará un fichero XML donde almacenará toda esta información sin cifrar. Estos ficheros pueden ser compartidos o privados.

Creación de una Activity que use SharedPreferences.

Realiza lo siguiente:

1) Para ver el uso de **SharedPreferences**, vamos a crear una Activity que tendrá los siguientes controles:

- **EditText** que será el correo electrónico de una persona (será el valor clave a almacenar)
- **EditText (multilínea)** datos a grabar
- **Botón** de grabar
- **Botón** de recuperar
- **Space**
- **Botón** de volver a la Activity Principal

2) Crea una nueva Activity (por ejemplo Preferencias. Y el **activity_preferencias** puede ser similar a lo siguiente:

NOTA: Es un ejemplo lo puedes diseñar a tu gusto (en las dos imágenes).

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".Preferencias">

    <TextView
        android:id="@+id/tvpreferencias"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center_horizontal"
        android:text="Preferencias"
        android:textSize="20sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/etemail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:hint="email"
        android:inputType="textEmailAddress" />

    <EditText
        android:id="@+id/etdatos"
        android:layout_width="match_parent"
        android:layout_height="400dp"
        android:ems="10"
        android:gravity="start|top"
        android:inputType="textMultiline" />
```

```
<LinearLayout
    android:orientation="horizontal">

    <Button
        android:id="@+id/btgrabar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Grabar" />

    <Button
        android:id="@+id/btrecuperar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Recuperar" />

</LinearLayout>

<Space
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_weight="1" />

<Button
    android:id="@+id/btvolver"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    android:gravity="start|top"
    android:text="Volver" />

</LinearLayout>
```

3) Una vez creado el fichero xml genera el código para trabajar las preferencias. Después de acceder a los componentes que hemos definido en el fichero xml. Puedes utilizar **findViewById** pero recuerda que también puedes utilizar **ViewBinding**. Una vez realizado el acceso a los componentes. A continuación obtenemos referencia al archivo de preferencias (en el ejemplo aparece agenda) pero puedes utilizar el nombre que tu quieras

```
// Obtenemos la referencia al archivo de preferencias
val preferencias = getSharedPreferences("agenda", Context.MODE_PRIVATE)
```

Obtenemos las preferencias utilizando el identificador (en el ejemplo agenda) y el Context que puede ser:

- **MODE_PRIVATE**. Sólo nuestra aplicación tiene acceso a estas preferencias.
- **MODE_WORLD_READABLE**. Todas las aplicaciones pueden leer estas preferencias, pero sólo la nuestra puede modificarlas.
- **MODE_WORLD_WRITEABLE**. Todas las aplicaciones pueden leer y modificar estas preferencias.

En nuestro caso por razones de seguridad hemos apostado por **MODE_PRIVATE**.

4) Cuando el usuario pulse el botón **grabar** vamos a crear un objeto de la clase al que vamos a llamar editor, para ello utilizamos un método del objeto preferencias.

```
// Definimos el objeto editor para poder acceder a las preferencias  
val editor = preferencia.edit()
```

5) Utilizaremos ese objeto para poder grabar en el archivo preferencias el correo electrónico que pongamos en el EditText correspondiente como clave y lo que tengamos en el EditText datos los datos asociados. Deberías comprobar que los EditText no estén vacíos. A continuación, puedes ver un ejemplo, de como grabar, el control sobre los EditText lo debes implementar tú:

```
// Almacenamos el valor  
editor.putString(etemail.text.toString(), etdatos.text.toString())
```

6) Finalmente confirmamos la grabación en el archivo de preferencias llamando al método **apply** en documentación también puedes ver que se puede realizar con un **commit** la diferencia entre utiliza uno u otro es la siguiente:

- **commit()** escribe los datos sincrónicamente (bloqueando el subproceso desde el que se llama). Entonces informa sobre el éxito de la operación.
- **apply()** programa los datos que se escribirán asincrónicamente. Es no informa sobre el éxito de la operación.

```
// Confirmamos la grabación  
editor.apply()
```

7) Por último, Mostramos en un Toast que se ha guardado el valor y limpiamos los valores introducidos.

```
// Mostramos un mensaje de confirmación de que se han guardados los datos  
Toast.makeText(context, this@Preferencias, text: "Datos guardados", Toast.LENGTH_LONG).show()  
// Limpiamos el contenido de los EditText  
etemail.text.clear()  
etdatos.text.clear()
```

8) Cuando el usuario pulse el botón **recuperar** vamos a intentar recuperar de las preferencias el valor que ha introducido en el EditText donde almacenamos el correo electrónico. Utilizamos un **getString** porque hemos almacenado texto si hubiésemos almacenado otro tipo de dato podríamos haber intentado recuperar un Int , Float, etc.

```
// Intentamos recuperar los datos asociados a email introducido  
val dat = preferencia.getString(etemail.text.toString(), defValue: "")
```

9) Luego comprobamos que no devuelva null (para evitar que devuelva ese valor). Y a continuación si data tiene cero caracteres luego podemos mostrar al operador que no hay un correo ingresado: Y en caso contrario mostramos los datos recuperados en el **etdatos**

```
if (dat!=null) {
    if (dat.length==0) {
        Toast.makeText( context: this@Preferencias, text: "No existen datos asociados a ese email", Toast.LENGTH_LONG).show()
    } else {
        etdatos.setText(dat)
    }
}
```

10) Comprueba la parte de la App que has realizado hasta ahora. En el siguiente video puedes ver un ejemplo de funcionamiento de lo realizado hasta ahora.

https://drive.google.com/file/d/1Dct35mZEnEZiBzHp6Knyu_BbASkFnasd/view?usp=sharing

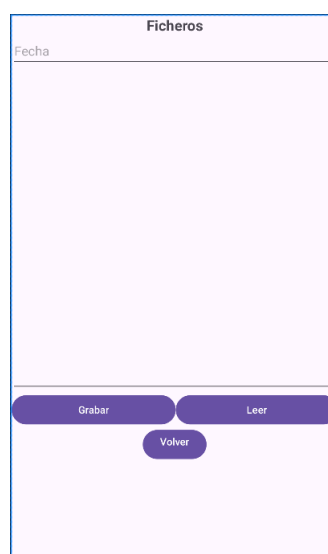
Ficheros.

Otra posibilidad de almacenar datos en nuestro dispositivo Android es el empleo de un archivo de texto que se guardará en el almacenamiento interno del equipo

11) Para ello vamos a crear una Activity donde vamos a almacenar en un fichero de texto (cuyo nombre será una fecha introducida por el usuario) las notas sobre ese día. Para ello utilizaremos los siguientes controles.

- **EditText** de tipo "Date" donde se carga la fecha. Luego otro
- **EditText** de tipo "Text Multiline"
- Tres **botones** (uno para grabar el fichero y otro para leerlo o recuperar contenido). Y otro para volver a la Activity Principal

12) A continuación puedes ver un ejemplo del aspecto de la aplicación y en la siguiente imagen de la Activity .



13) También puedes ver el contenido del fichero xml con la vista.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".Ficheros">

    <TextView
        android:id="@+id/tv_fichero"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center_horizontal"
        android:text="Ficheros"
        android:textSize="20sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/et_fecha"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:hint="Fecha"
        android:inputType="date" />

    <EditText
        android:id="@+id/et_ficheros"
        android:layout_width="match_parent"
        android:layout_height="450dp"
        android:ems="10"
        android:gravity="start|top"
        android:inputType="textMultiLine" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <Button
            android:id="@+id/bt_fgrabar"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Grabar" />

        <Button
            android:id="@+id/bt_read"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Leer" />

    </LinearLayout>

    <Space
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:id="@+id/bt_home"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:gravity="start|top"
        android:text="Volver" />

</LinearLayout>
```

14) Una vez creado el fichero xml pasamos generar el código para trabajar con los ficheros. En primer lugar, debes acceder a los componentes que hemos definido en el fichero xml. Puedes utilizar **findViewById** o **ViewBinding**.

15) Cuando el usuario pulsa el botón **Guardar** primero tenemos que extraer la fecha ingresada en el primer EditText (en el ejemplo **etfecha**) y remplazar las barras de la fecha por guiones ya que no se puede utilizar este carácter dentro de un nombre de archivo en Android. Si quieres en vez de poner el código completo podrías definir un método (o función) que lo realice y realizar llamada en el **setOnClickListener**

```
val nombrefichero = etfecha.text.toString().replace(oldChar: '/', newChar: '-')
```

16) A continuación como has realizado en el módulo de AED para leer ficheros utilizando un **try { } catch { }**. Para ello creamos un objeto **OutputStreamWriter** y al constructor de dicha clase le enviamos el dato que retorna el método **openFileOutput** propio de la clase **AppCompatActivity** que le pasamos como parámetro el nombre del archivo de texto y el modo de apertura. Luego llamamos al método **write** y le pasamos el String a grabar, en este caso extraemos los datos del EditText. Luego de grabar con el método **write** llamamos al método **flush** para que vuelque todos los datos que pueden haber quedado en el buffer y cerramos el archivo.

```
try {  
    val fichero = OutputStreamWriter(openFileOutput(nombrefichero, Activity.MODE_PRIVATE))  
    fichero.write(etfidatos.text.toString())  
    fichero.flush()  
    fichero.close()  
    Toast.makeText(context: this@Ficheros, text: "Fichero Guardado.", Toast.LENGTH_LONG).show()  
} catch (e: IOException) {  
    Toast.makeText(context: this@Ficheros, text: "Error de lectura", Toast.LENGTH_LONG).show()  
}
```

17) Cada vez que se graba un dato se genera un archivo de texto para dicha fecha, si ya había un archivo con el mismo nombre (es decir la misma fecha) se pisa el anterior.

18) Cuando el usuario pulsa el botón **Leer**, lo primero que hacemos es como cuando escribíamos el fichero, también tenemos que extraer la fecha ingresada en el primer EditText (en el ejemplo **etfecha**) y remplazar las barras de la fecha por guiones ya que no se puede utilizar este carácter dentro de un nombre de archivo en Android. Si quieres en vez de poner el código completo podrías definir un método (o función) que lo realice y realizar llamada en el **setOnClickListener**

19) Luego comprobamos si el archivo existe recuerda que en este caso estamos poniendo como nombre del archivo la fecha.

```
if (fileList().contains(nomfich)) {
```

20) Si existe el archivo lo abrimos, leemos y cargamos el **etfidatos** con los datos del archivo.

```
try {  
    val fich = InputStreamReader(openFileInput(nomfich))  
    val buffer = BufferedReader(fich)  
    var linea = buffer.readLine()  
    val contenidototal = StringBuilder()  
    while(linea!=null) {  
        contenidototal.append(linea+"\n")  
        linea = buffer.readLine()  
    }  
    buffer.close()  
    fich.close()  
    etfidatos.setText(contenidototal)  
} catch (e: IOException) {}
```

21) Utilizamos un objeto de la clase **InputStreamReader**. Creamos un objeto de la clase **BufferedReader** para hacer más eficiente la lectura del archivo. Luego dentro del bucle vamos leyendo línea a línea el archivo de texto y la guardamos en un objeto de la clase **StringBuilder**. Los objetos String son inmutables. Con cada concatenación usando el operador + o el método plus() , obtenemos un nuevo objeto String . Por el contrario, para evitar la creación innecesaria de objetos String , podemos usar un **StringBuilder**. Por lo tanto, **StringBuilder** crea un solo búfer interno que contiene la cadena final. Por lo tanto, **StringBuilder** es más eficiente al concatenar una gran cantidad de cadenas

22) Mostramos los datos recuperados en el EditText correspondiente (en nuestro ejemplo **etfidatos**).

23) A continuación puedes observar en un video la ejecución de la aplicación.

<https://drive.google.com/file/d/1sfMBV3JlykUk0HW55UNC8JcupHPG7x3d/view?usp=sharing>